



UMT

Database Systems

Unit 6 – Entity Relationship Modeling

School of Systems & Technology - SST

Chapter Objectives

In this chapter you will learn:

- How to use Entity–Relationship (ER) modeling in database design.
- The basic concepts associated with the ER model: entities, relationships, and attributes.
- A diagrammatic technique for displaying an ER model using the Unified Modeling Language (UML).
- How to identify and resolve problems with ER models called connection traps.

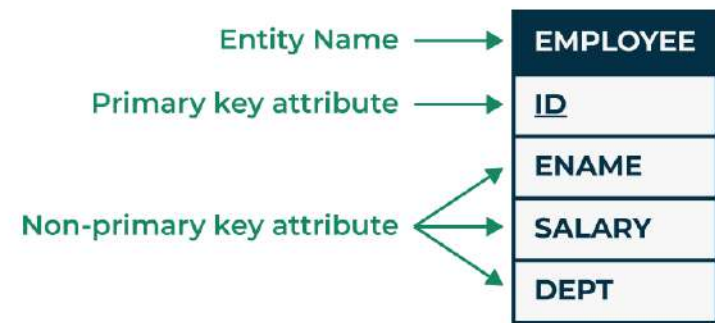
Entity / Entity Types

Entity Type: A group of objects with the same properties and independent existence.

This is the blueprint or template. For example, STUDENT is the entity type. It defines the structure (schema) that all students will follow.

Entity (Instance) or Entity Occurrence: is a uniquely identifiable object of an entity type. In the table, the individual rows (like "Andrew" or "Isabella") are the entities.

Entity Set: The collection of all entities of a particular type at a specific point in time (the entire table of students).

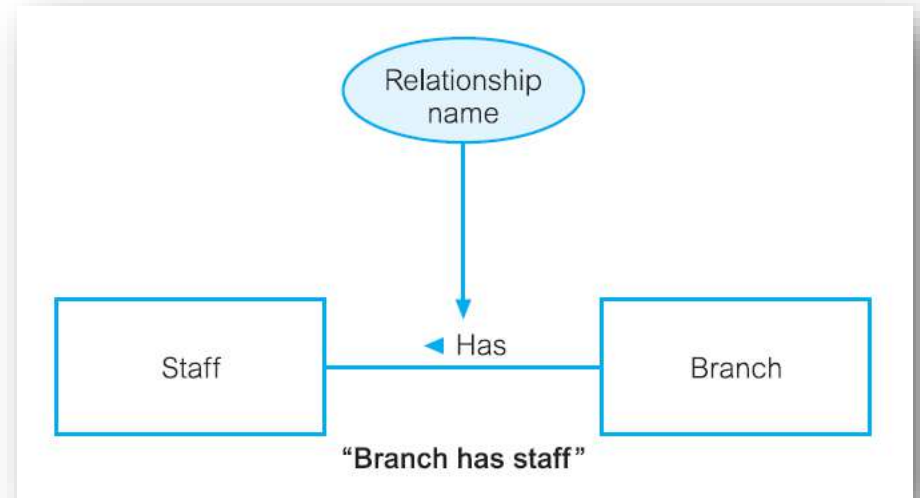


Student				Entity Type
Roll_no	Student_name	Age	Mobile_no	
1	Andrew	18	7089117222	→ E1
2	Oliver	19	8709054568	→ E2
3	Isabella	20	9864257315	→ E3
4	Julian	21	9847852156	→ E4

ENTITY SET
E1
E2
E3
E4

Relationship Types

- **Relationship Type:** It describes a meaningful association between two Entity Types (like PROFESSOR and COURSE). It's the "verb" that connects them.
- Example: "Professors Teach Courses."
- We usually represent this with a line or a diamond shape connecting two boxes.
- **Relationship Occurrence:** This is one specific instance of that relationship. It's when a specific professor actually gets linked to a specific course in the real world.
- Example: "Faisal Teaches Data Structures."
- In a Database: This would be a specific entry in a table that connects Professor ID 101 with Course ID CS202.



Relationship Types

The Ovals: The ovals represent Entity Sets.

- Left Oval represents The set of all Branches (the offices).
- Right Oval represents The set of all Staff (the employees).
- Middle Oval represents The set of all Relationships currently happening between them.

The Points inside the Ovals: Each point represents an Entity Instance (also called an Entity Occurrence).

- It is NOT an attribute. An attribute would be something like "Branch Name" or "Staff Salary."
- It IS a specific object. B003 is one specific branch office. SG37 is one specific person.

The Middle Oval (r1, r2, r3): These points represent Relationship Occurrences.

- Each *r* is a connection line.
- *r1* is the specific "link" that tells the database, "Staff SG37 works at Branch B003."

The Logic of the Lines: The lines reveal the Rules of the business:

- Look at B003: Two lines come out of it (r1 and r2). This means one branch can have multiple staff members.
- Look at the Staff (Right side): Only one line goes into each person. This means one staff member can only work at one branch at a time.

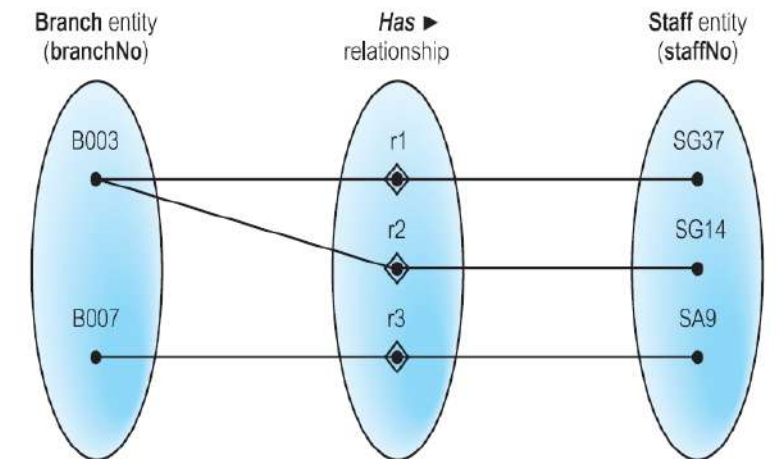
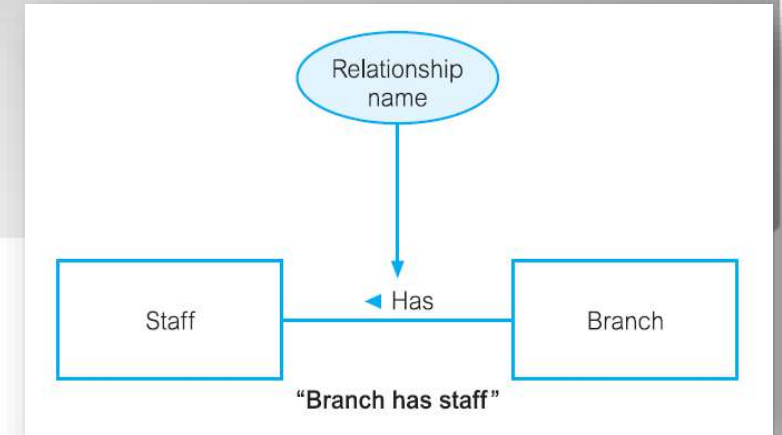
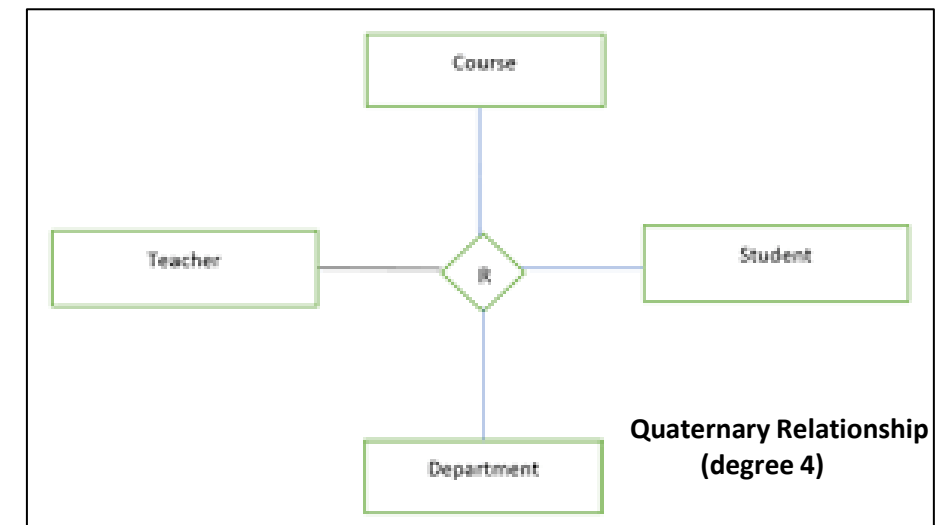
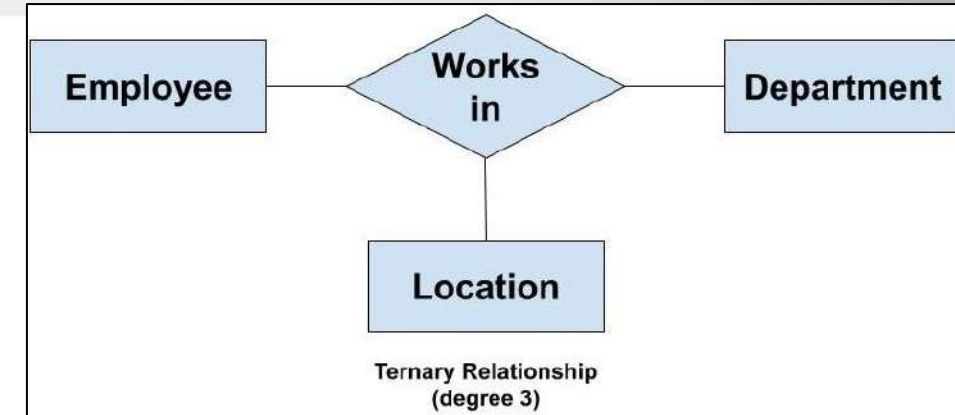
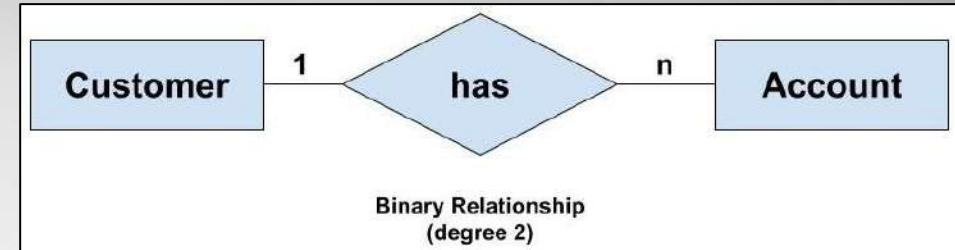
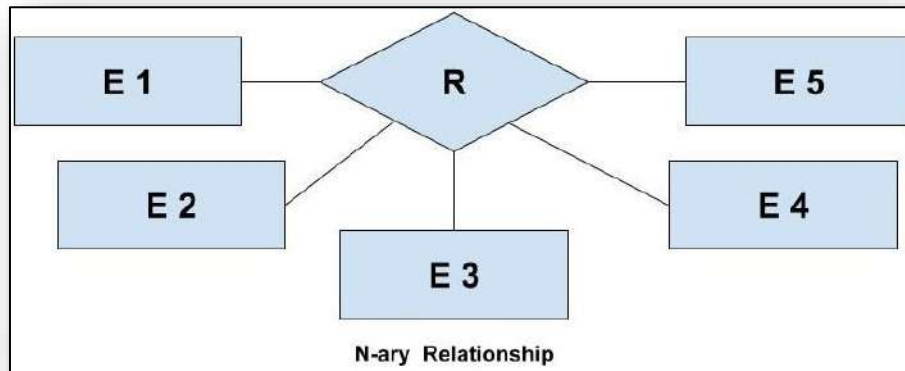


Figure 12.4 A semantic net showing individual occurrences of the *Has* relationship type.

Degree of a relationship type

- ❖ The **degree of a relationship type** is the number of participating entity types in a relationship.
- A relationship of degree two is called **binary**.
- A relationship of degree three is called **ternary**.
- A relationship of degree four is called **quaternary**.



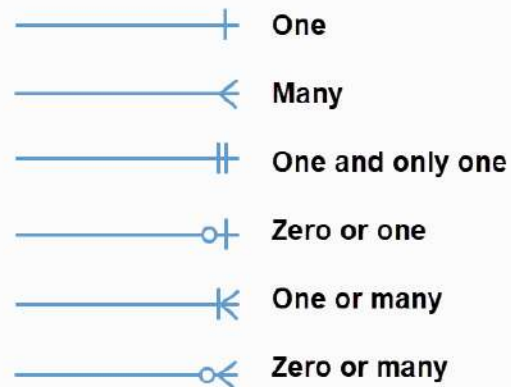
Cardinality of a Relationship Type

Cardinality: The number of instances of one entity type that can be associated with each instance of another entity type.

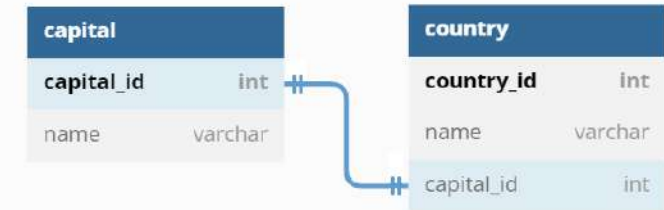
One-to-One (1:1): One instance of A is associated with exactly one instance of B. Example: One Manager manages one Department.

One-to-Many (1:N): One instance of A is associated with many instances of B. Example: One University has many Students.

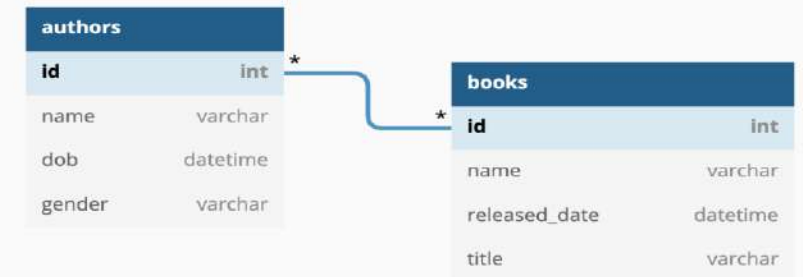
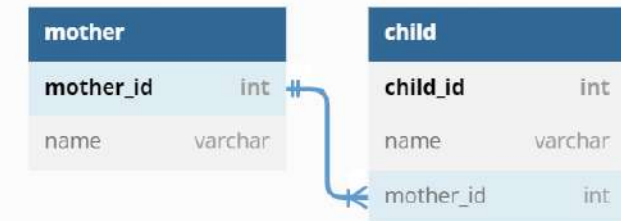
Many-to-Many (M:N): Many instances of A are associated with many instances of B. Example: Many Students enroll in many Courses.



One to one database relationship



One to many database relationship

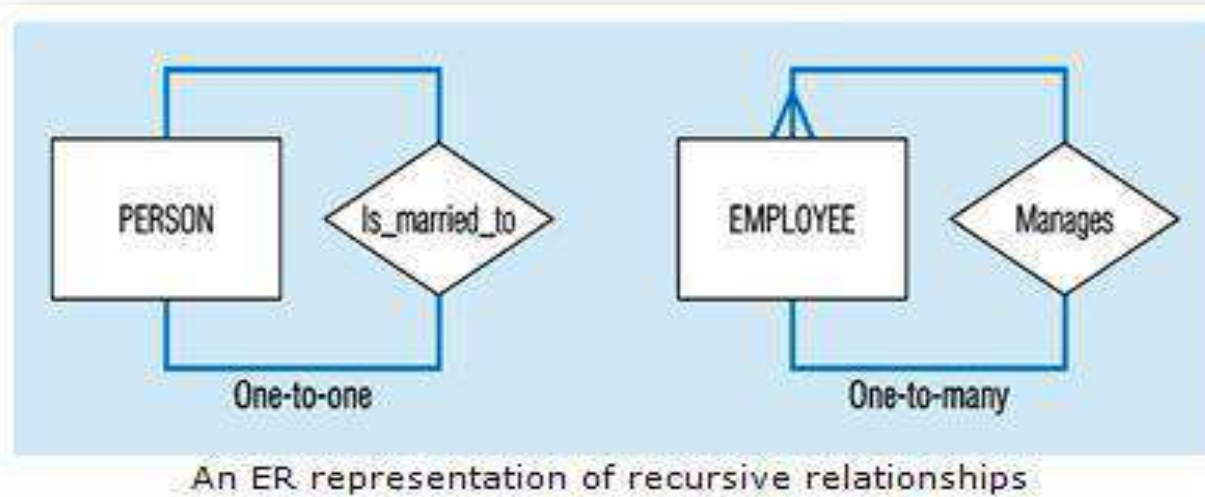
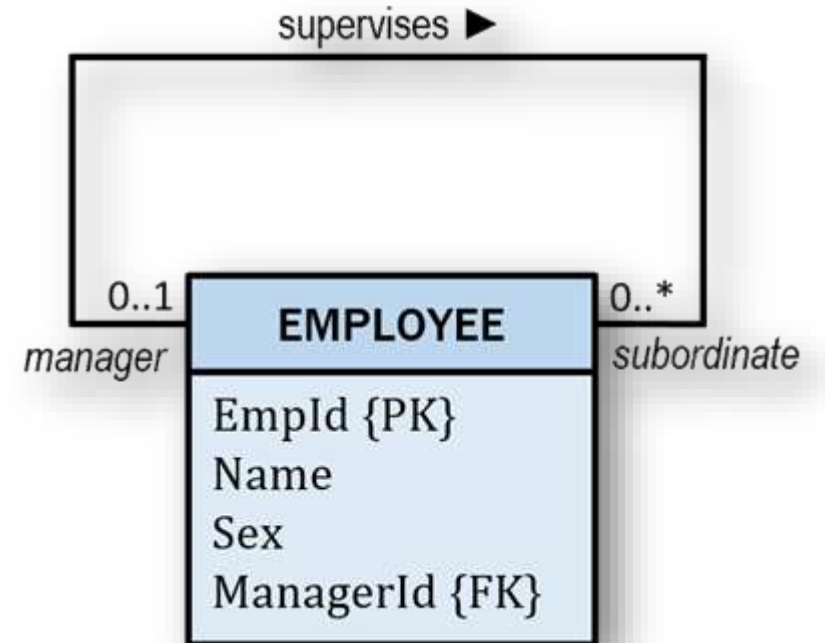


Recursive (Unary) Relationship

Definition: A relationship where an entity type associates with itself.

Roles: The same entity plays different parts (e.g., an Employee acts as a Manager to another Employee).

Implementation: Uses a Self-Referencing Foreign Key (one column in the table points back to the Primary Key of the same table).



Associative Entity

- An **associative entity** (sometimes called a "bridge" or "junction" table) is an entity used to resolve a **Many-to-Many (M:N)** relationship between two other entities.
- A many-to-many relationship cannot be represented directly in a relational model. It must be **decomposed into two one-to-many relationships** using an associative entity. This design avoids redundancy and maintains data integrity.
- An associative entity should be created when:
 - We have a **Many-to-Many** relationship between two tables.
 - The relationship has attributes that cannot logically belong to either entity alone.
 - The relationship is independent enough to be treated as its own "thing" or "event."

Associative Entity

Real-World Example: Student Enrollment

- Imagine a database for a University:
 - A **Student** can enroll in many **Courses**.
 - A **Course** can have many **Students**.
- **The Problem:** Where do you store the **Grade**?
 - If you put it in the Student table, you don't know which course it's for.
 - If you put it in the Course table, you don't know which student earned it.
- **The Solution:** Create an associative entity called **Enrollment**.

Associative Entity

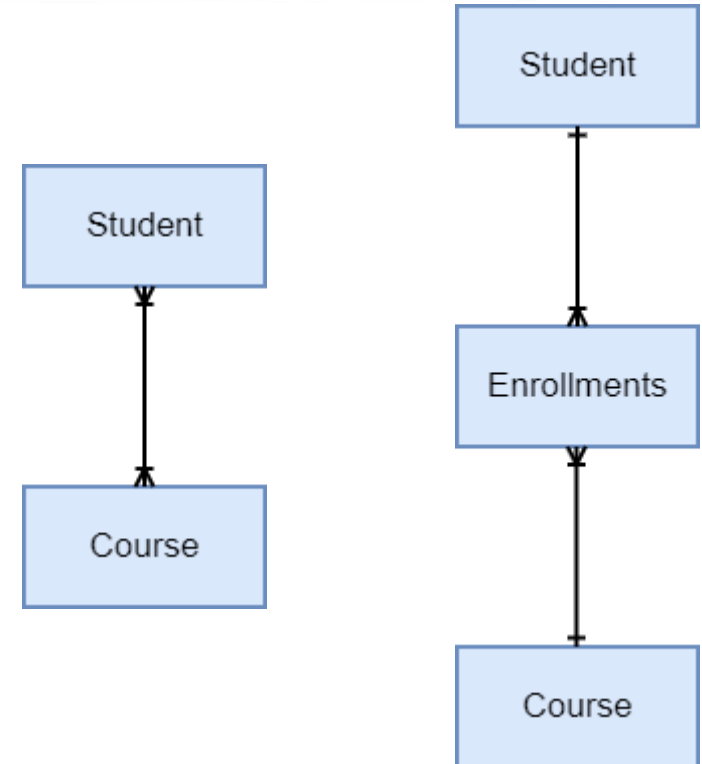
Real-World Example: Student Enrollment

- Imagine a database for a University:
 - A **Student** can enroll in many **Courses**.
 - A **Course** can have many **Students**.
- The Problem:** Where do you store the **Grade**?
 - If you put it in the Student table, you don't know which course it's for.
 - If you put it in the Course table, you don't know which student earned it.

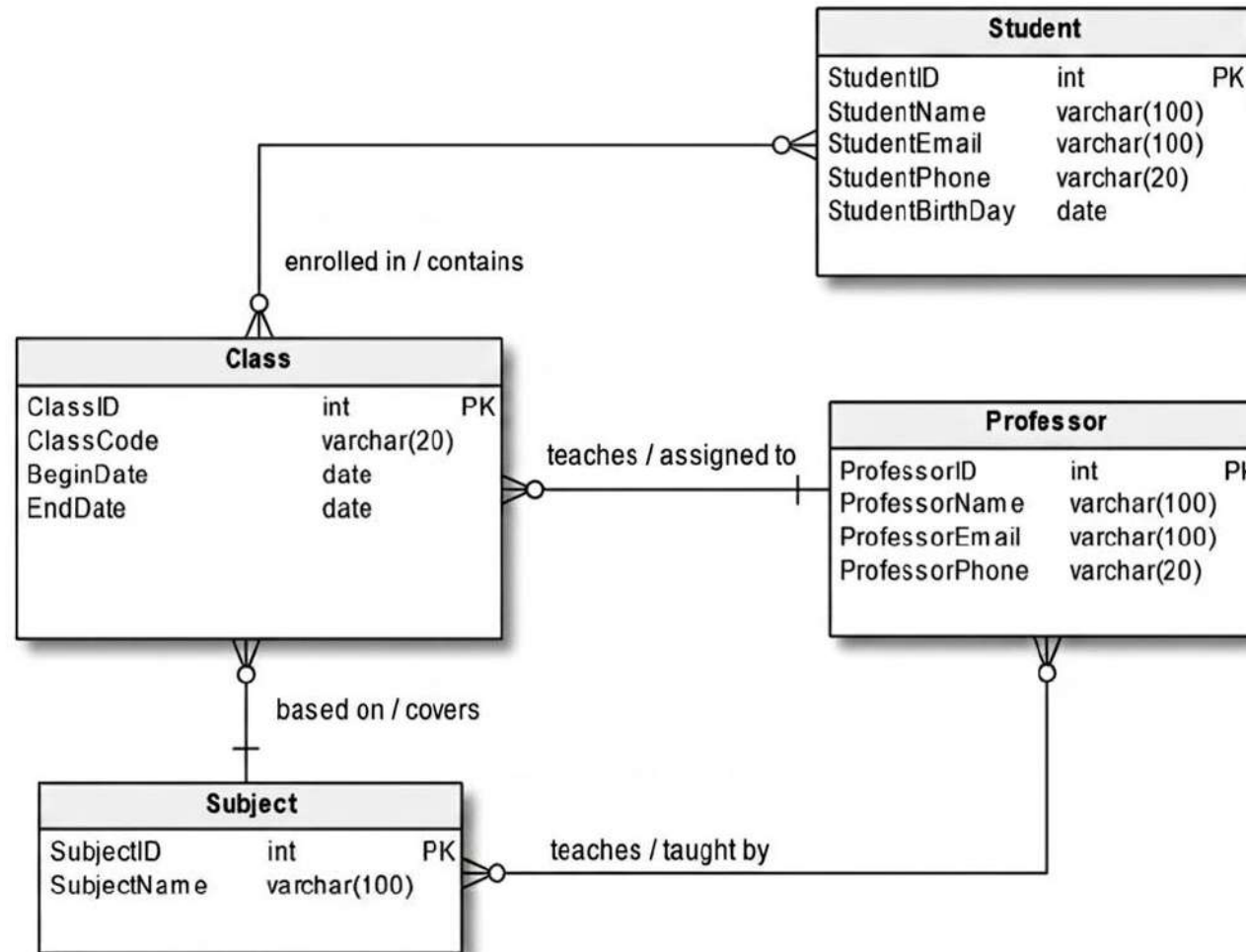
- The Solution:** Create an associative entity called **Enrollment**.

Key Characteristics

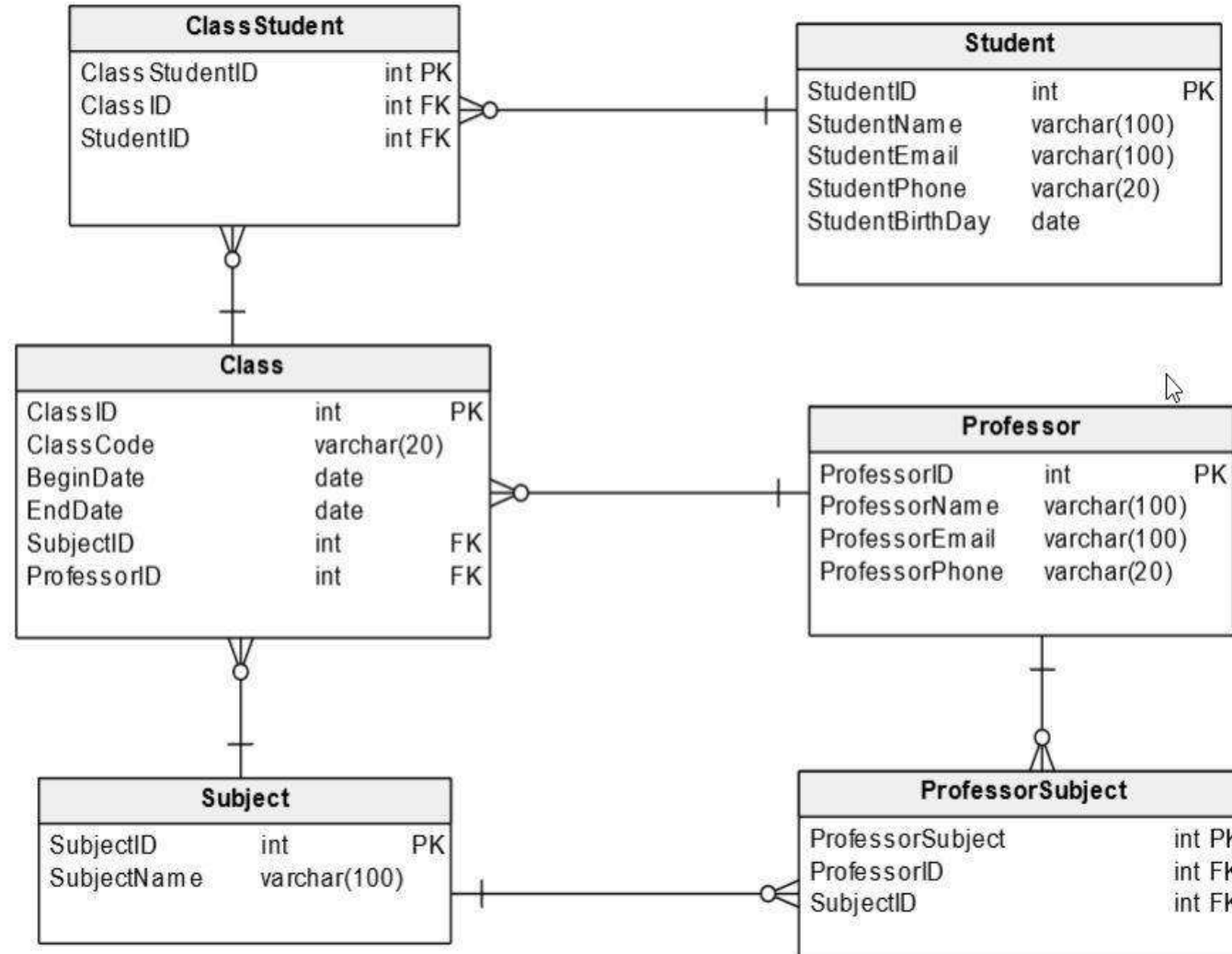
- Relationship Link:** It contains Foreign Keys (FK) from both parent entities.
- Primary Key Options:**
 - A **Composite Primary Key** made of both Foreign Keys.
 - A new, separate column created specifically to identify each row (e.g., Enrollment_ID).



Academic System – ER Diagram



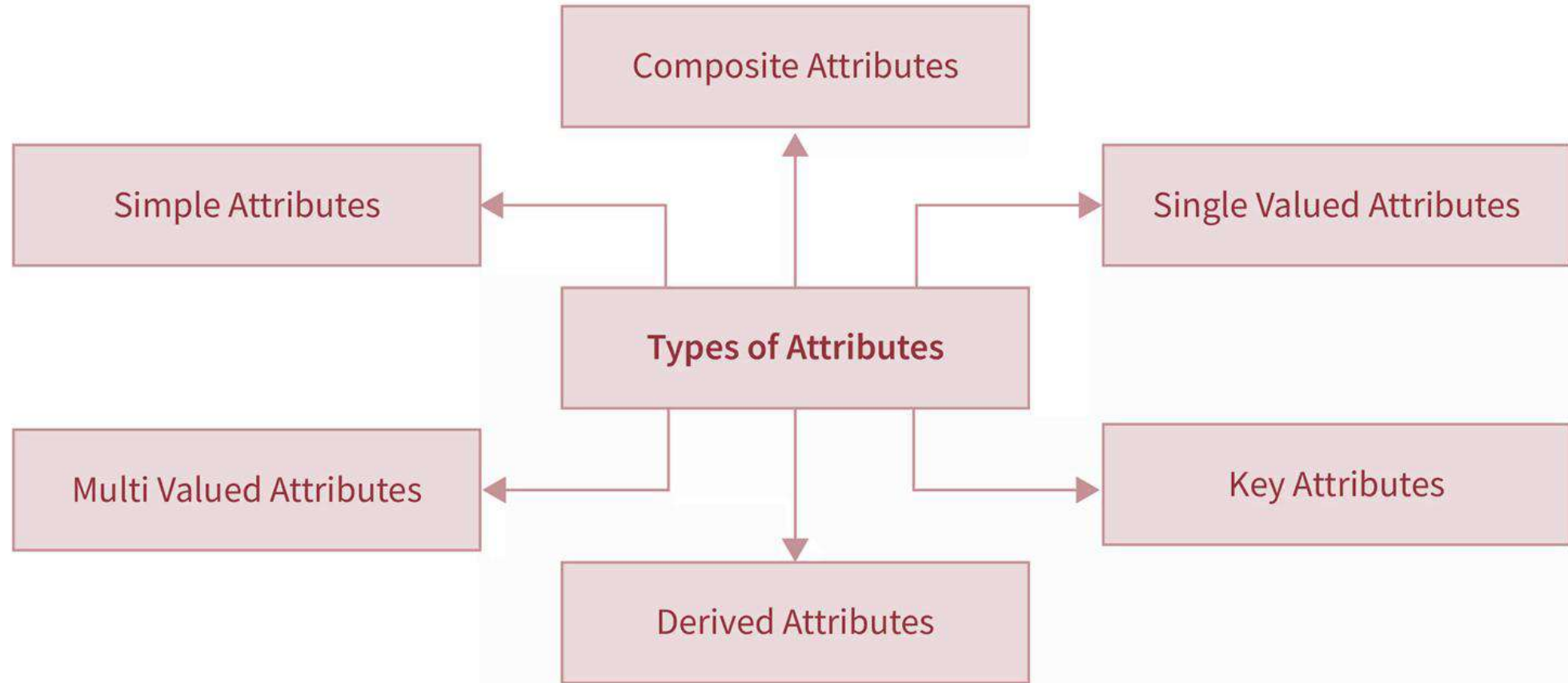
Academic System – ER Diagram (Corrected)



Attributes

- An attribute is a **property or characteristic of an entity**.
- An entity may contain **any number of attributes**.
- **One of the attributes is considered as the primary key**. In an Entity-Relation model, attributes are represented in an elliptical shape.
- Example: Student has attributes like name, age, roll number, and many more. To uniquely identify the student, we use the primary key as a roll number as it is not repeated. Attributes can also be subdivided into another set of attributes.
- There are six types of attributes: Simple, Composite, Single-valued, Multi-valued, and Derived. A sixth, though rarely used, type is the Complex attribute.

Types of Attributes



Types of Attributes in ER Model

Simple attribute :

- An attribute that cannot be further subdivided into components is a simple attribute.
- Example: The roll number of a student, the gender of a person.

Composite attribute :

- An attribute that can be split into smaller sub-parts is a composite attribute.
- Example: The address can be further split into house number, street number, city, state, country, and pin code, the name can also be split into first name, middle name, and last name.

Single-valued attribute :

- The attribute which takes up only a single value for each entity instance is a single-valued attribute.
- Example: The age of a student.

Contd....

Multi-valued attribute :

- The attribute which takes up more than a single value for each entity instance is a multi-valued attribute.
- Example: Phone number of a student, or skills of a person.

Derived attribute :

- An attribute that can be derived from other attributes is derived attributes.
- Example: Age can be derived from Date of Birth.

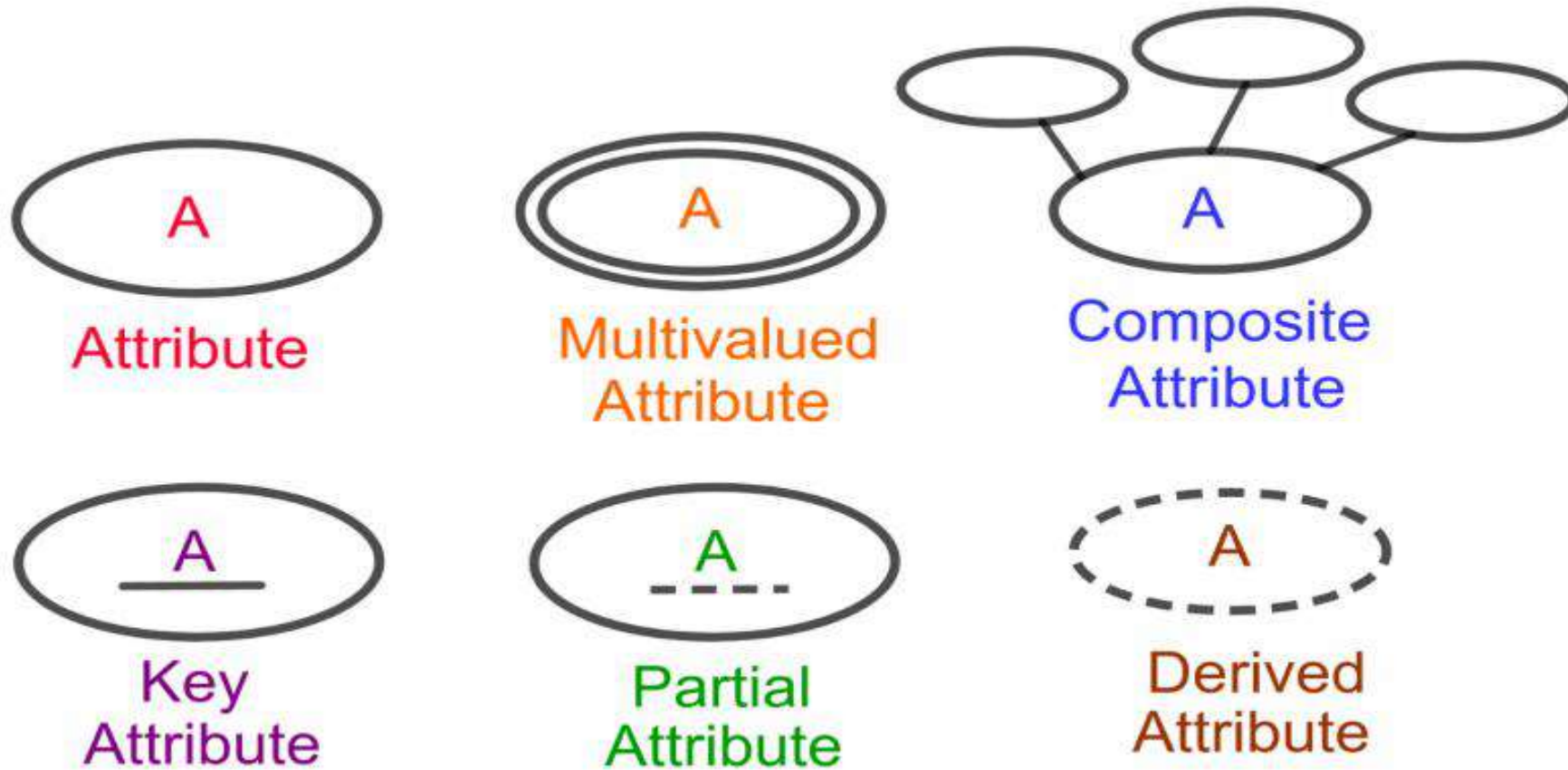
Key attribute:

- Key attributes are those attributes that can uniquely identify the entity in the entity set.
- Example: Roll-No is the key attribute because it can uniquely identify the student.

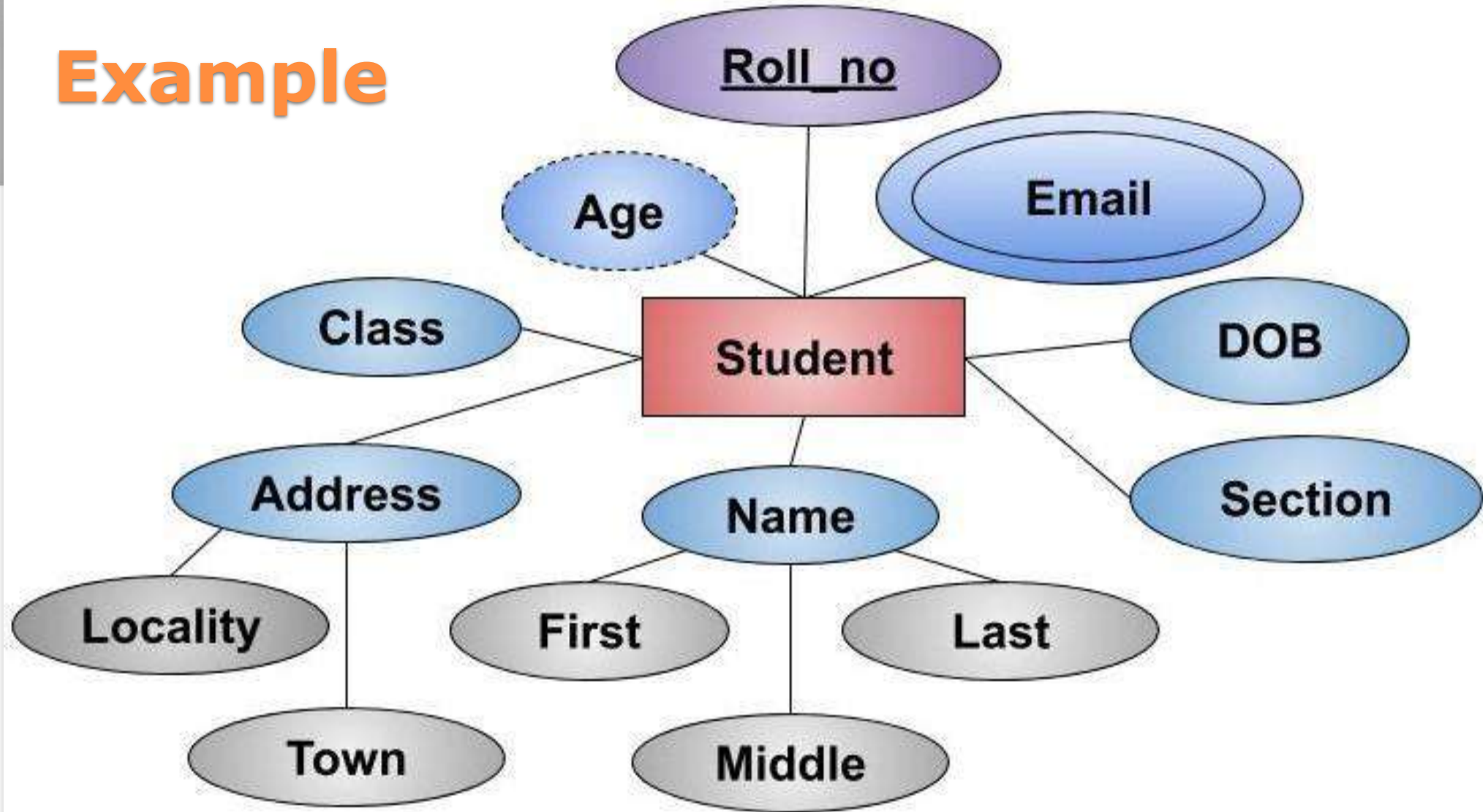
Complex Attribute:

- It is a combination of composite and multi-valued attributes (e.g., an "Address" attribute that is composite, but a person can have "Multiple Addresses").

How attributes are represented



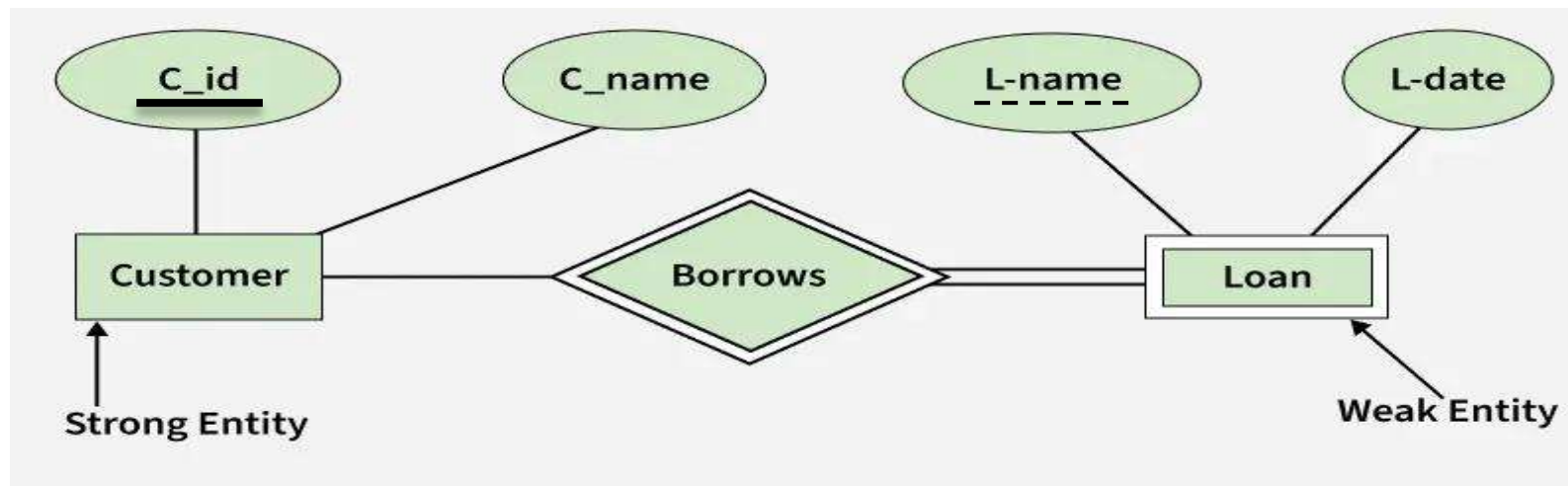
Example



Strong vs Weak Entity Types

Strong Entity

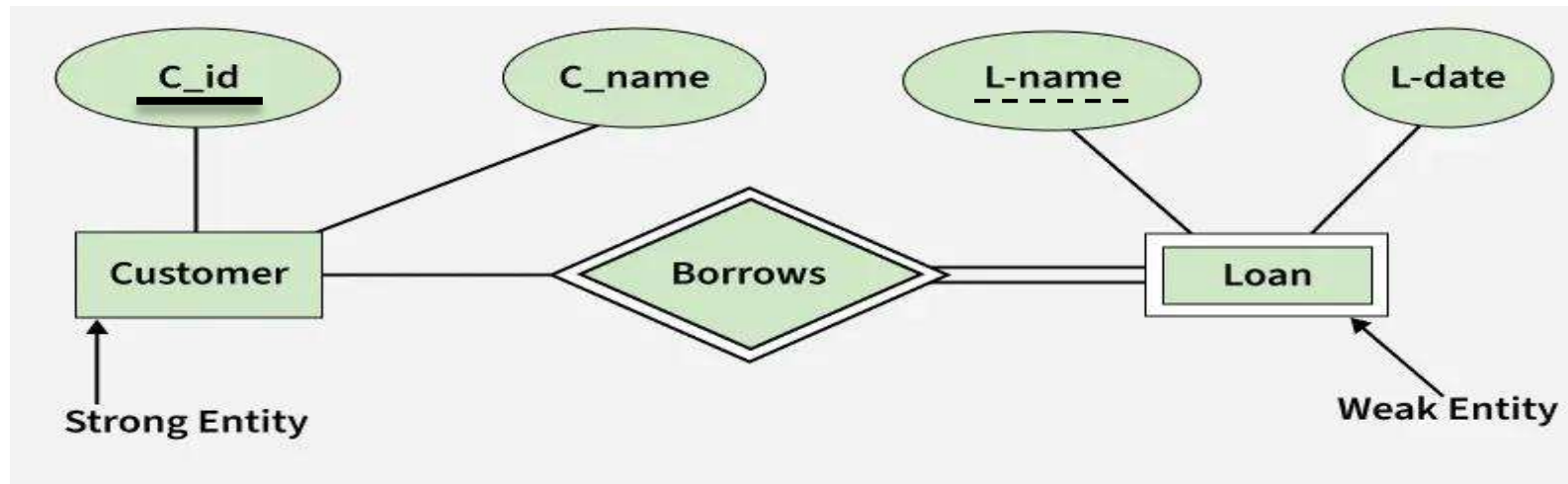
- An entity that exists independently and can be uniquely identified on its own. It is **not** dependent on any other entity.
- **Key Identification:** It always possesses a **Primary Key** (underlined with a solid line).
- **Example from Diagram (Customer):**
- The "**Customer**" entity is strong.
- It is identified by its own unique primary key: C_id.
- The existence of a "Customer" does not depend on whether they have a "Loan" or not.
- **ER Notation:** Represented by a **single rectangle** (like "Customer").



Strong vs Weak Entity Types

Weak Entity: An entity whose existence is dependent upon another "strong" (owner) entity. It cannot be uniquely identified by its own attributes alone.

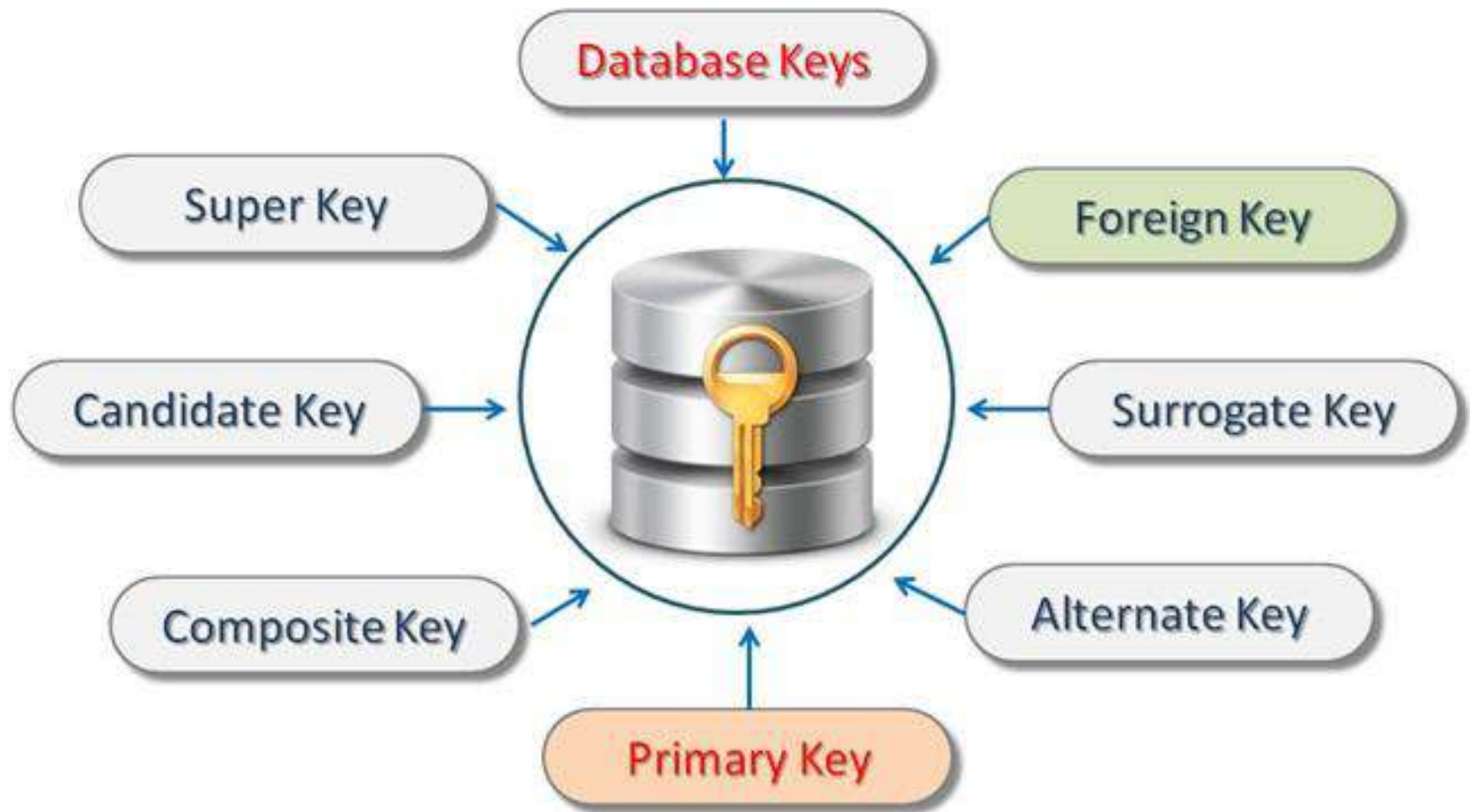
- **Key Identification:** It does not have a primary key. It relies on a partial attribute (Discriminator), which is visually represented by a **dashed underline**. To uniquely identify a record, the Discriminator is combined with the owner entity's primary key ($C_id + L_name$).
- **Example from Diagram (Loan):**
- The "**Loan**" entity is weak.
- Its existence is tied directly to a "**Customer**" through the "**Borrows**" identifying relationship (double diamond).
- Notice that **L-name** has a **dashed underline**, marking it as the partial key. A specific loan is identified by knowing the C_id and the specific L-name. If you delete the "Customer," all their associated "Loans" must be deleted too.
- **ER Notation:** Represented by a **double rectangle** (like "Loan").



Database Keys

- A key **refers to an attribute/a set of attributes that help us identify a row** (or tuple) uniquely in a table (or relation).
- A key is **also used when we want to establish relationships** between the different columns and tables of a relational database.
- The individual values present in a key are commonly referred to as **key values**.
- We use a key for **defining various types of integrity constraints** in a database.
- A table, on the other hand, represents a collection of the records of various events for any relation.
- The keys in DBMS **can be a combination of multiple attributes** (or columns), or they can be just one single attribute.
- The primary motive of the keys is to **provide every record with a unique identity** of its own.

DB Key types



Types of Keys

1. Primary Key

- The primary key refers to a column or a set of columns of a table that helps us identify all the records uniquely present in that table. A table can consist of just one primary key. Also, this primary key cannot consist of the same values reappearing/repeating for any of its rows. All the values of a primary key have to be different, and there should be no repetitions.

2. Super Key

- A super key refers to the set of all those keys that help us uniquely identify all the rows present in a table. It means that all of these columns present in a table that can identify the columns of that table uniquely act as the super keys.

Types of Keys

3. Candidate Key

- The candidate keys refer to those attributes that identify rows uniquely in a table. In a table, we select the primary key from a candidate key. Thus, a candidate key has similar properties as that of the primary keys that we have explained above. In a table, there can be multiple candidate keys.

4. Alternate Key

- As we have stated above, any table can consist of multiple choices for the primary key. But, it can only choose one. Thus, all those keys that did not become a primary key are known as alternate keys.

5. Foreign Key

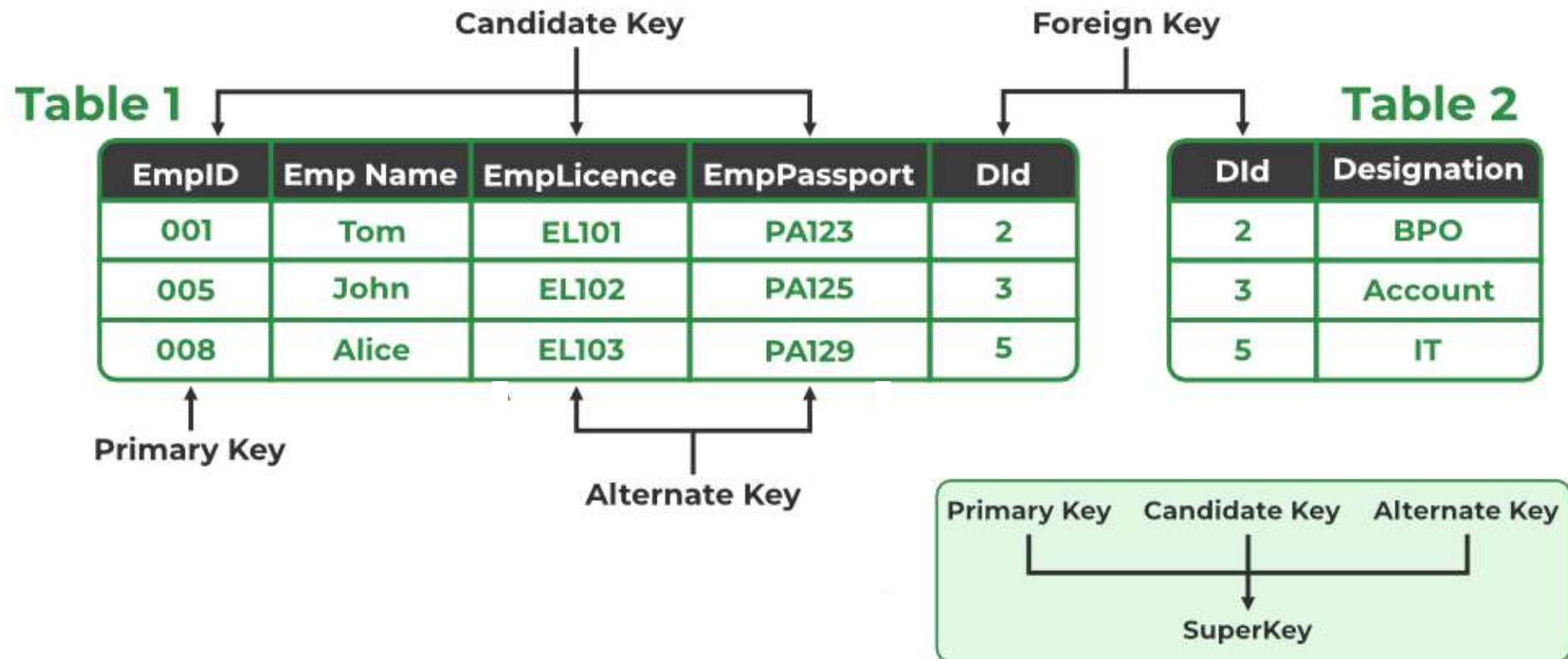
- We use a foreign key to establish relationships between two available tables. The foreign key would require every value present in a column/set of columns to match the referential table's primary key. A foreign key helps us to maintain data as well as referential integrity.

Types of Keys

6. Composite Key

- The composite key refers to a set of multiple attributes that help us uniquely identify every tuple present in a table. The attributes present in a set may not be unique whenever we consider them separately. Thus, when we take them all together, it will ensure total uniqueness.

Examples



Foreign Key — What is a Foreign Key (FK)

A **Foreign Key** is a column (or group of columns in case of composite primary key) in the **Child table** that points to the **Primary Key** of the **Parent table**. It acts as a cross-reference between tables because it references the primary key of another table.

Key Terminology:

- **Referencing (Child) Table:** The table containing the Foreign Key.
- **Referenced (Parent) Table:** The table containing the candidate key (usually the Primary Key) being pointed to.

Table: Departments (Parent)

Dept_ID (PK)	Dept_Name	Building_Name
101	Artificial Intelligence	SST-1
102	Data Science	SST-1

Table: Employees (Child)

Emp_ID (PK)	Emp_Name	Dept_ID (FK)
1	Dr. Smith	101
2	Prof. Khan	101
3	Ms. Lee	102

Foreign Key — The Logic of the Link

- A Foreign Key (FK) is used to connect one table to another.
- It points to a Primary Key (PK) because a PK is always unique (no duplicates).
- If we tried to link Employees to a non-unique column like Building_Name, and two departments were in the same building, the database wouldn't know which department Dr. Smith actually belongs to.
- By using a PK, the database always knows exactly which row we mean.
- This makes sure every record in the child table is linked to one clear, correct record in the parent table.

Table: Departments (Parent)

Dept_ID (PK)	Dept_Name	Building_Name
101	Artificial Intelligence	SST-1
102	Data Science	SST-1

Table: Employees (Child)

Emp_ID (PK)	Emp_Name	Dept_ID (FK)
1	Dr. Smith	101
2	Prof. Khan	101
3	Ms. Lee	102

Foreign Key — Summary of Rules

- **Data Type Match:** If Dept_ID in the Parent table is an INT, the Dept_ID in the Child table **must** also be an INT.
- **Domain Consistency:** The values in the Employees. Dept_ID column must either match a value in Departments. Dept_ID or be NULL (if the employee hasn't been assigned a department yet).
- **Logical Ownership:** The Foreign Key always lives in the table that represents the "Many" side of the relationship (Multiple employees belong to one department).

Table: Departments (Parent)

Dept_ID (PK)	Dept_Name	Building_Name
101	Artificial Intelligence	SST-1
102	Data Science	SST-1

Table: Employees (Child)

Emp_ID (PK)	Emp_Name	Dept_ID (FK)
1	Dr. Smith	101
2	Prof. Khan	101
3	Ms. Lee	102

Foreign Key — Modeling Relationships

One-to-Many (1:N)

Concept: One record in the "Parent" table relates to many in the "Child" table.

Logic: Place the Foreign Key on the **Many** side.

Example: Departments and Professors

- One **Department** can have many **Professors**.
- One **Professor** works for exactly one **Department**.

Departments (Parent)			
DeptID (PK)	DeptName	Building	Budget
101	Computer Science	Turing Hall	\$500,000
102	Mathematics	Euler Annex	\$350,000

Professors (Child)			
ProfID (PK)	Name	Salary	DeptID (FK)
P_01	Dr. Smith	\$95,000	101
P_02	Dr. Jones	\$88,000	101
P_03	Dr. Brown	\$91,000	102

Foreign Key — Modeling Relationships

One-to-One (1:1)

Concept: Each record in Table A is related to **at most one** record in Table B, and vice versa.

Logic: Place the FK in either table, but apply a **UNIQUE constraint** so a value cannot be repeated.

Example: Professors and Office Spaces

- One **Professor** is assigned one **Office**.
- One **Office** belongs to only one **Professor**.

If you try to assign P_01 to a second room, the database will throw an error because of the Unique rule on the FK.

Professors		
ProfID (PK)	Name	Email
P_01	Dr. Smith	smith@uni.edu
P_02	Dr. Jones	jones@uni.edu

OfficeAssignments			
OfficeNum (PK)	SquareFootage	PhoneExt	ProfID (FK + UNIQUE)
Room 404	120	x4404	P_01
Room 502	150	x4502	P_02

Foreign Key — Modeling Relationships

Many-to-Many (N:M)

The Concept: Multiple records in Table A relate to multiple records in Table B.

The Rule: Databases cannot do this directly. We must use an **Associative (Junction) Table** that holds FKs from both sides.

Example: Students and Courses

- A **Student** enrolls in many **Courses**, a **Course** has many **Students**.

StudentID (PK)	FirstName	Age	Phone Number
S_99	Alice	20	033XXXXXXXX
S_88	Bob	21	034XXXXXXXX

CourseID (PK)	Title	Credits
CS101	Database I	3
MA202	Calculus II	4

StudentID (FK)	CourseID (FK)	Grade	Semester
S_99	CS101	A	Fall 2025
S_99	MA202	B+	Fall 2025
S_88	CS101	A-	Spring 2026

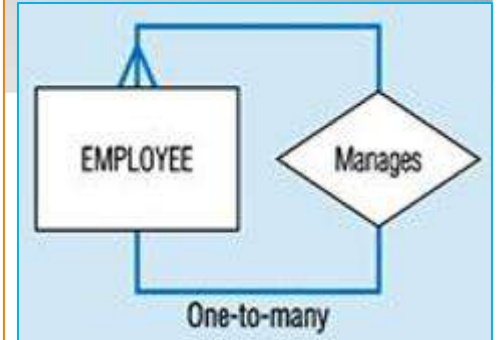
Foreign Key — Modeling Relationships

Self-Referencing (Unary)

The Concept: A table where a record relates to another record within the same table.

The Rule: The FK points back to the PK of the **same table**.

- **Example: Employee Management Hierarchy** In this scenario, a Manager (who is also an employee) supervises other employees.
- **E_02 and E_03** both report to **E_01**.
- **ManagerID** is a Foreign Key that "points" back to the **EmployeeID** column.
- **NULL** is used for the employee at the top of the hierarchy (e.g., the CEO) who has no manager.



EmployeeID (PK)	Name	Job Title	ManagerID (FK)
E_01	Alice Chen	CEO	NULL
E_02	Bob Harris	VP of Sales	E_01
E_03	Charlie Wu	VP of Tech	E_01
E_04	David Rossi	Sales Lead	E_02

Foreign Keys — Referential Integrity

Departments (Parent)	
Dept_ID (PK)	Dept_Name
101	Artificial Intelligence
102	Data Science

Professors (Child)		
Prof_ID	Prof_Name	Dept_ID (FK)
1	Dr. Faizan	101
2	Dr. Sana	102

Referential Integrity (RI): The guarantee that every Foreign Key (FK) in a "Child" table always points to a valid, existing Primary Key (PK) in the "Parent" table.

The Rules (The Restrictions):

The database prevents three types of data corruption:

- **INSERT:** You cannot add a Child (Professor) pointing to a non-existent Parent (Dept ID: 999).
- **DELETE:** You cannot delete a Parent (Dept) if Children (Professors) are still linked to it.
- **UPDATE:** You cannot change a Parent PK without "fixing" the Child FKs first.

Foreign Keys — Referential Integrity

Departments (Parent)	
Dept_ID (PK)	Dept_Name
101	Artificial Intelligence
102	Data Science

Professors (Child)		
Prof_ID	Prof_Name	Dept_ID (FK)
1	Dr. Faizan	101
2	Dr. Sana	102

When a user tries to **Update** or **Delete** a **Parent** record, the database doesn't *have* to just block the action. We can define specific **Policies** for how the database should react:

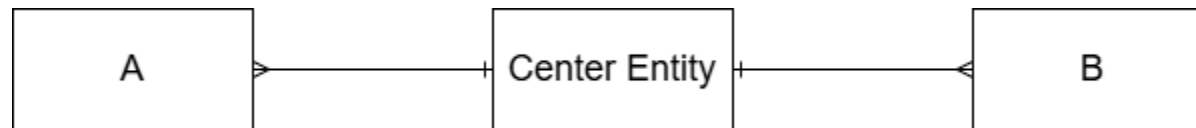
Policy	Result
RESTRICT (No Action)	The default. The database blocks the deletion/update and throws an error.
CASCADE	If the Parent is deleted, the database automatically deletes all linked Child records.
SET NULL	If the Parent is deleted, the Child's Foreign Key is changed to NULL .
SET DEFAULT	If the Parent is deleted, the Child is moved to a pre-defined default value.

Problems with ER Models — Fan Trap

Introduction to the Fan Trap

- **The Concept:** A Fan Trap occurs when a model represents two 1:N relationships linked to a central entity, but fails to show the association between the two "end" entities.

- **The Structure:**



- **The Symptom:** You can link A to Center and B to Center, but you cannot link A to B with certainty.

Problems with ER Models — Fan Trap

- Imagine a University database where we track which **Department** a **Staff** member belongs to, and which Department offers a **Course**.
- **Relationship 1:** A Department employs many Staff members.
- **Relationship 2:** A Department offers many Courses.
- **The Logical Gap:** We know the Department for both, but we don't know which Staff member is assigned to which Course.



Problems with ER Models — Fan Trap



Question: Who teaches Quantum Mechanics?

Answer: Someone in Department D01... but we don't know if it's Dr. Bohr or Dr. Fermi." (This is the Trap!)

Department	
Dept_ID	Dept_Name
D01	Physics

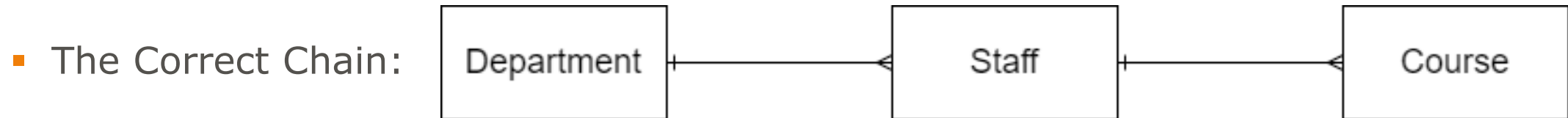
Course		
Course_ID	Title	Dept_ID (FK)
PH101	Quantum Mechanics	D01
PH102	Thermodynamics	D01

Staff		
Staff_ID	Name	Dept_ID (FK)
S1	Dr. Bohr	D01
S2	Dr. Fermi	D01

Problems with ER Models — Fan Trap

Solution to Fan Trap

- The most semantically correct fix is to place the *active* entity in the middle.



- Logic:
 - The Department employs the Staff
 - The Staff member is responsible for the Course.
- Result: We know which Staff works for which Dept. We know exactly who teaches which Course. We can still find the Course's Department by following the chain back through the Staff member.

Solution to the Fan Trap



Question: Who teaches Quantum Mechanics?

Answer: Dr. Bohr teaches Quantum Mechanics

Department	
Dept_ID	Dept_Name
D01	Physics

Course		
Course_ID	Title	Staff_ID (FK)
PH101	Quantum Mechanics	S1
PH102	Thermodynamics	S2

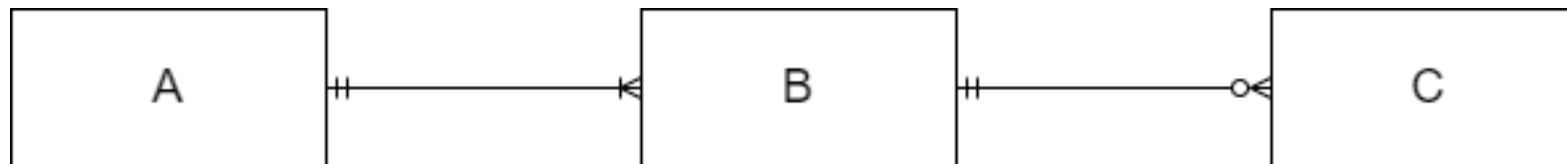
Staff		
Staff_ID	Name	Dept_ID (FK)
S1	Dr. Bohr	D01
S2	Dr. Fermi	D01

Problems with ER Models — Chasm Trap

Introduction to the Chasm (Pronounced Ka.zm)Trap

The Concept: A Chasm Trap occurs when a model suggests a relationship between entities through an intermediate entity, but the pathway "breaks" because some records are not mandatory.

- **The Structure:** A relationship exists between Entity A and Entity C via Entity B, but because the link between A and B (or B and C) is **optional**, we lose the connection to Entity C for certain records.



- **The Symptom:** You can see some connections, but others "fall into the chasm" and disappear from your queries.

Problems with ER Models — Chasm Trap

- In this scenario, we assume that every Course is linked to a Department through a Staff member.
- **Relationship 1:** A Department employs many Staff (Mandatory: Every Staff must belong to a Dept).
- **Relationship 2:** A Staff member oversees many Courses (**Optional:** A Staff member might not be teaching any course right now).
- **The Trap:** If we want to see all Courses offered by a Department, we can only see the ones currently assigned to a Staff member.



Problems with ER Models — Chasm Trap

Let's look at what happens when a Course exists but hasn't been assigned a Professor yet.

The Question: “List all courses offered by the Physics Department (D01).”

The Data's Answer: “Quantum Mechanics.”

The Reality: Thermodynamics is also a Physics course, but because it has no Staff_ID assigned, it is "lost" in the chasm between Department and Course.

Dept_ID	Dept_Name
D01	Physics

Staff_ID	Name	Dept_ID (FK)
S1	Dr. Bohr	D01

Course_ID	Title	Staff_ID (FK)
PH101	Quantum Mechanics	S1
PH102	Thermodynamics	NULL



Solution to the Chasm Trap

Solution: Add a direct (1:N) relationship between Department and Course.

1. We link **Course** to **Department** directly (Mandatory link). We now know PH102 belongs to D01.
2. We keep the link between **Staff** and **Course** (Optional link). We use this only to see who is teaching what.

Outcome: No more lost data! Even if a course has no teacher, we still know which department owns it.

Dept_ID	Dept_Name
D01	Physics

Staff_ID	Name	Dept_ID (FK)
S1	Dr. Bohr	D01

Course_ID	Title	Staff_ID (FK)	Dept_ID (FK)
PH101	Quantum Mechanics	S1	D01
PH102	Thermodynamics	NULL	D01





Thankyou

Any Queries?